

多模态大模型原理

关键词：多模态，CLIP，LLaVA，视觉语言模型，模态对齐，Whisper，Gemini

多模态大模型原理

关键词：多模态，CLIP，LLaVA，视觉语言模型，模态对齐，Whisper，Gemini

多模态学习的定义与意义

多模态学习 (Multimodal Learning)

是指机器能够同时处理、理解和生成两种或两种以上模态信息（如文本、图像、音频、视频、3D点云、传感器数据等）的学习范式。与单模态学习相比，多模态学习更接近人类感知世界的方式——人类天然地

多模态大模型 (Multimodal Large Language Model, MLLM) 的意义体现在

1. 信息互补：不同模态携带互补信息。例如图像提供空间与外观信息，文本提供抽象与语义信息，融合后能获得更全面、准确的理解。
2. 突破语义鸿沟：让模型能够将“猫”这个词与猫的视觉形象对应起来，弥合符号与感知之间的鸿沟。
3. 支撑通用智能：真正的人工通用智能 (AGI) 必须能够处理现实世界的多模态信息，单模态文本模型无法完整实现。
4. 拓展应用边界：从图像描述、视觉问答、文档理解到机器人控制、自动驾驶，多模态能力是众多实际应用的基础。

典型的多模态大模型由 模态编码器 (Modality Encoder)、模态对齐模块 (Alignment Module) 和大语言模型 (LLM) 三部分组成。

模态对齐方法

模态对齐 (Modality Alignment)

是多模态学习的核心难题——不同模态的数据分布、特征空间、语义粒度差异巨大，需要

1. 对比学习 (Contrastive Learning)

对比学习通过拉近正样本对、推远负样本对来学习跨模态的共享表征。CLIP 是这一范式的代表。其核心思想是：在批次内构造 $N \times N$ 的图文对，只有 N 个是真实匹配的正样本，其余 $N^2 - N$ 个为负样本，使用 InfoNCE 损失进行优化。

CLIP 风格的对比学习伪代码

```
import torch
import torch.nn as nn
```

```

import torch.nn.functional as F

class CLIP(nn.Module):
    def __init__(self, textencoder, imageencoder, dim=512):
        super().__init__()
        self.textencoder = textencoder
        self.imageencoder = imageencoder
        self.logitscale = nn.Parameter(torch.ones([]) * 4.0)

    def forward(self, texts, images):
        textfeat = F.normalize(self.textencoder(texts), dim=-1)
        imagefeat = F.normalize(self.imageencoder(images), dim=-1)
        logitscale = self.logitscale.exp()
        logits = logitscale * textfeat @ imagefeat.T # [B, B]
        labels = torch.arange(logits.size(0), device=logits.device)
        loss = (F.crossentropy(logits, labels) +
                F.crossentropy(logits.T, labels)) / 2
        return loss

```

2. 共享编码器 (Shared Encoder)

使用统一编码器 (如 `Transformer`) 同时处理多模态输入序列, 让不同模态的 `token` 在同一网络中交互。这种方式需要将图像、音频等转换为类似文本 `token` 的离散或连续表示。`ViLT` 等模型采用了类似思路。

3. 交叉注意力 (Cross-Attention)

通过 `Cross-Attention` 机制让一个模态的表征作为 `Query`, 另一个模态作为 `Key/Value`, 实现深层交互。`Flamingo`、`BLIP-2` 中的 `Q-Former` 即采用此机制。

`Q-Former` 中的交叉注意力伪代码

```

class QFormer(nn.Module):
    def __init__(self, numqueries=32, dim=768):
        super().__init__()
        self.queries = nn.Parameter(torch.randn(numqueries, dim))
        self.crossattn = nn.MultiheadAttention(dim, numheads=8)
        self.selfattn = nn.MultiheadAttention(dim, numheads=8)

```

```
def forward(self, imagefeatures):
# queries 作为可学习的 " 问题 "，从图像特征中提取信息
q = self.queries.unsqueeze(0).expand(imagefeatures.size(0), self.queries.size(1), self.queries.size(2))
q = self.selfattn(q, q, q)[0]
q = self.crossattn(q, imagefeatures, imagefeatures)
return q # 用于注入 LLM 的对齐表征
```

模态对齐方法对比

方法 代表模型 优点 缺点

对比学习 CLIP、ALIGN 训练简单、可扩展性强 仅对齐全局表征，缺乏细粒度交互

共享编码器 ViLT、OFA 架构统一、交互充分 训练成本高、模态间冲突

交叉注意力 BLIP-2、Flamingo 细粒度交互、灵活 架构复杂、推理开销大

视觉 - 语言模型 (VLM)

CLIP

CLIP (Contrastive Language - Image Pre-training) 由 OpenAI 提出，通过 4 亿对图文数据进行对比学习预训练。CLIP 开创了大规模视觉 - 语言预训练范式，其图像编码器 (ViT) 和文本编码器 (Transformer) 输出对齐到同一向量空间，支持零样本图像分类。CLIP 的图像编码器至今仍是众多多模态模型 (如 LLaVA、Stable Diffusion) 的视觉骨干。

BLIP / BLIP-2

BLIP (Bootstrapping Language - Image Pre-training) 通过自举机制清洗噪声数据。BLIP-2 进一步提出 Q-Former，将冻结的视觉编码器与冻结的 LLM 桥接，显著降低训练成本，是参数高效多模态对齐的里程碑。

LLaVA

LLaVA (Large Language and Vision Assistant) 采用极简架构，视觉特征通过一个 MLP 投影层直接拼接进 LLM 的词嵌入序列，与文本 token 一起送入 LLM。这证明了 " 投影层 + 指令微调 " 即可获得强大的多模态对话能力，是开源多模态模型的重要基线。

LLaVA 风格的多模态输入构造

```
def buildllavainput(image, textprompt, visionencoder):
# 1. 提取图像 patch 特征
```

```

imagefeatures = visionencoder(image) # [Npatches, D]
# 2. 投影到 LLM 词嵌入空间
imagetokens = projector(imagefeatures) # [Npatches, D]
# 3. 文本 token 化
texttokens = llm.tokenize(textprompt) # [Ntext, D]
# 4. 拼接: [IMG] <imagetokens> [/IMG] <texttokens>
inputembeds = torch.cat([imagetokens, texttokens])
return llm.generate(inputembeds=input_embeds)

```

GPT-4V

GPT-4V 是 OpenAI 首个原生支持视觉输入的 GPT 模型，展现出强大的图文推理、文档理解、图表分析能力。GPT-4V 能够处理复杂视觉推理、少样本学习，并能识别图像中的文字与逻辑关系，是多模态大模型商业化的标杆。

音频 - 语言模型

Whisper

Whisper 是 OpenAI 开源的多语言语音识别模型，在 68 万小时多语言音频数据上训练，支持 100 多种语言的转录与翻译。Whisper 采用 Encoder-Decoder Transformer 架构，对噪声和口音有较强鲁棒性，是当前语音识别领域的事实标准之一。

AudioPaLM

AudioPaLM

将音频与文本统一为同一语义空间，能够同时处理语音识别、文本转语音、语音到语音翻译等多种任务。其核心是 audio token，与文本 token 共享同一个 Transformer 进行建模，实现真正的 "听 - 说 - 写"。

音频多模态应用场景

- 语音识别 (ASR) 与语音翻译 (AST)
- 文本转语音 (TTS) 与语音克隆
- 语音情感识别与说话人识别
- 音乐生成与音频修复

视频理解与生成

视频理解需要建模时序信息，常见方法包括：

- 3D CNN：如 C3D、I3D，通过 3D 卷积捕获时空特征
- Video Transformer：如 TimeSformer、ViViT，将时间维度引入注意力机制
- Video-LLM：如 Video-LLaVA、Video-ChatGPT，将视频帧序列编码后与 LLM 结合

视频生成方面，Sora、Kling（可灵）、Runway Gen-3 等模型采用 Diffusion Transformer (DiT) 架构，通过时空 patch 化、自回归或扩散过程生成高质量长视频，

统一多模态架构

Gemini

Gemini 是 Google 提出的原生多模态模型，从一开始就在多模态数据上联合训练，支持文本、图像、音频、视频输入输出。与 "先 LLM 再接视觉编码器" 的拼接式方案不同，Gemini 强调端到端的原生多模态训练。

GPT-4o

GPT-4o 中的 "o" 代表 "omni"，是 OpenAI 首个真正的全模态模型，能在单一神经网络中处理文本、图像、音频输入并输出文本、图像、音频，端到端延迟仅 200ms，支持实时语音对话，是统一多模态架构的代表性成果。

拼接式 vs 原生多模态对比

维度 拼接式 (LLaVA、BLIP-2) 原生 (Gemini、GPT-4o)

训练方式 先 LLM 再接视觉编码器 多模态联合预训练

模态交互 早期融合较浅 深层跨模态交互

延迟 较高 较低

扩展性 模块化、易迭代 整体重训成本高

多模态评估挑战

多模态模型评估面临独特挑战：

1. 评估基准多样：VQA、GQA、MMBench、MMMU、SEED-Bench 等评测维度各异
2. 主观性强的任务：图像描述、视频生成质量难以客观打分
3. 幻觉问题：模型可能描述图像中不存在的细节 (Object Hallucination)
4. 组合泛化：模型对空间关系、属性绑定等组合推理能力仍弱
5. 跨模态对齐质量：缺乏统一度量

常用评估方法包括 POPE（对象存在性评估）、CHAIR（幻觉率）、MMMU（大学级多学科理解）以及主观评分。

应用场景

- 图像描述（Image Captioning）：自动生成图像描述，用于无障碍辅助、内容检索
- 视觉问答（VQA）：根据图像回答自然语言问题，应用于教育、医疗影像分析
- 文档理解（Document Understanding）：解析图表、表格、公式，如 GPT-4o
- 机器人控制（Robotics）：RT-2、PaLM-E 将视觉输入转化为机器人动作指令
- 自动驾驶：多模态感知融合摄像头、激光雷达数据
- 医疗 AI：结合影像、病历、检验报告进行辅助诊断
- 内容创作：图文配对、视频脚本生成、广告创意

总结

多模态大模型正从 " 视觉编码器 + LLM " 的拼接式架构走向原生统一架构。模态对齐是核心技术挑战，对比学习、交叉注意力、共享编码器各有优势。从 CLIP 到 GPT-4o，多模态模型的能力边界不断拓展，正在重塑人机交互、内容创作、具身智能等众多领域。未来研